# EE434 Programming Project

John Welgoss

50518667

EE434

April 14, 2026

## Overview

This project implements all four tasks outlined in the assignment using python, numpy, and matplotlib. All non-uniform random variables are generated from uniform U[0,1] samples. The queueing questions are solved using event-driven simulation rather than plugging formulas into code.

Questions covered:

- Q1: Uniform random number generation and estimation of P(U > x)
- Q2: Exponential and Poisson random variable generation from the uniform generator
- Q3: Event-driven M/M/1 queue simulation
- Q4: Event-driven $M/E_K/1$ queue simulation

## Methods

Q1: UniformRandomGenerator class generates i.i.d. sample s in [0,1]. To estimate P(U > x), the code generates many samples and counts the fraction greater than x.

Q2:

- Exponential samples are generated with inverse transform sampling : X = -mean * ln(U)
- Poisson samples are generated from uniforms using Knuth's product algorithm.

This satisfies the requirement that only the uniform random generator may be used as the source of randomness.

Q3: The M/M/1 queue is simulated with an event-driven engine. Arrival and departure events are scheduled on a priority queue. The simulator collects:

- Time average number of packets in the system, E[N]
- Packet average delay, E[T]
- Steady state occupancy probabilities, $P_n$

The first part of the simulation is discarded as warmup to reduce initialization bias.

Q4: The $M/E_K/1$ queue uses the same event driven approach. Service times are generated as <u>Erlang random variables:</u>

$$E_K = x_1 + x_2 + \ldots + x_K$$

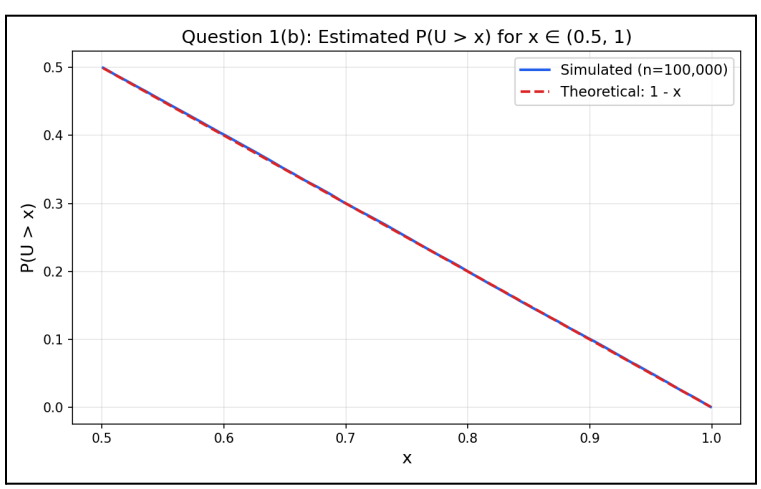
Where each phase is exponential and all phases are generated from uniform samples through the Q2 exponential generator.

**Results and Discussion**

Q1: The estimated P(U > x) curve is approximately a straight line over x in (0.5, 1), which matches the theoretical result:

$$P(U > x) = 1 - x$$

Matplotlib figure:

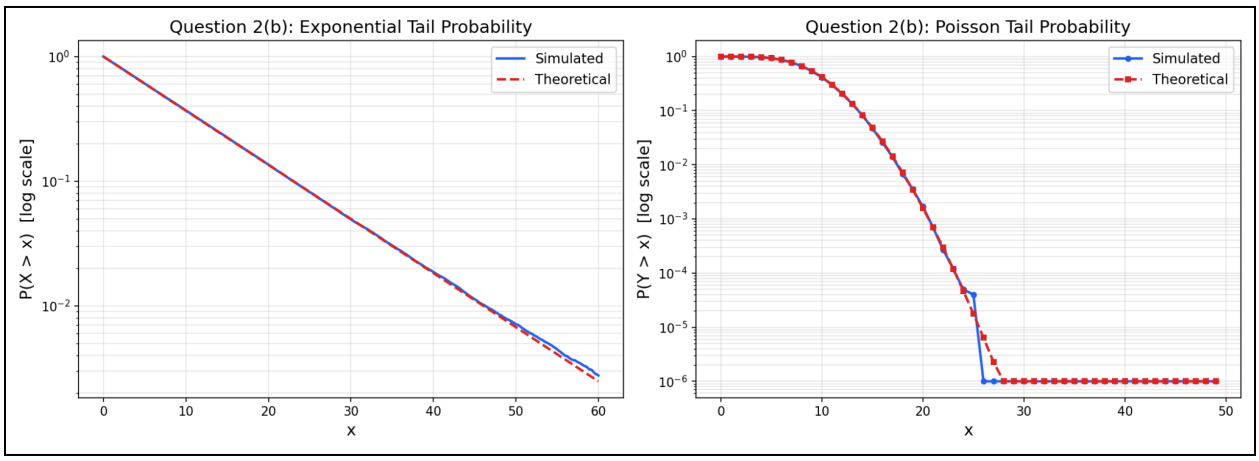


Q2: For E(x) = E(y) = 10, the generated samples matched the target means in the project run:

- Exponential sample mean ~ 10.0314
- Poisson sample mean ~ 9.9771

On a semilog plot, the exponential tail appears close to a straight line, while the poisson tail decreases more mildly.

Matplotlib figure:




Q3: For $\lambda = 5$ and $\mu = 6$, the simulation produces $P_n$ values that closely follow the geometric M/M/1 steady-state distribution. In the end-to-end run:

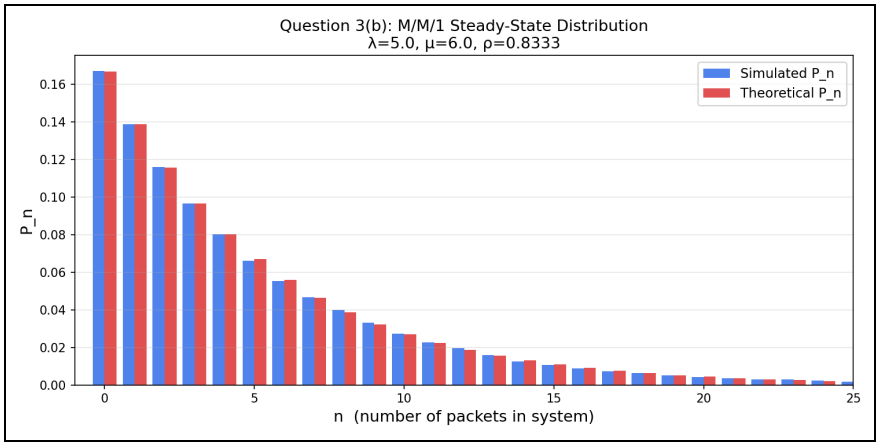
- Simulated E[N] = 5.1293
- Theoretical E[N] = 5.0000
- Simulated E[T] = 1.024481
- Theoretical E[T] = 1.000000

Little's Law also matched closely as a validation

- $E[N] / \lambda = 1.025867$
- Simulated E[T] = 1.024481

These values were collected directly from simulation rather than derived from Little's Law

Matplotlib figure:




Notes:

- The simulation directly tracks arrivals, service completions, and packet delays
- E[N] is measured as a time average
- E[T] is measured by averaging packet sojourn times

Q4: For k = 4, $\lambda = 5$, and $\mu = 6$, the $M/E_4/1$ queue has a smaller expected number of packets than M/M/1 under the same utilization. In the end-to-end run:

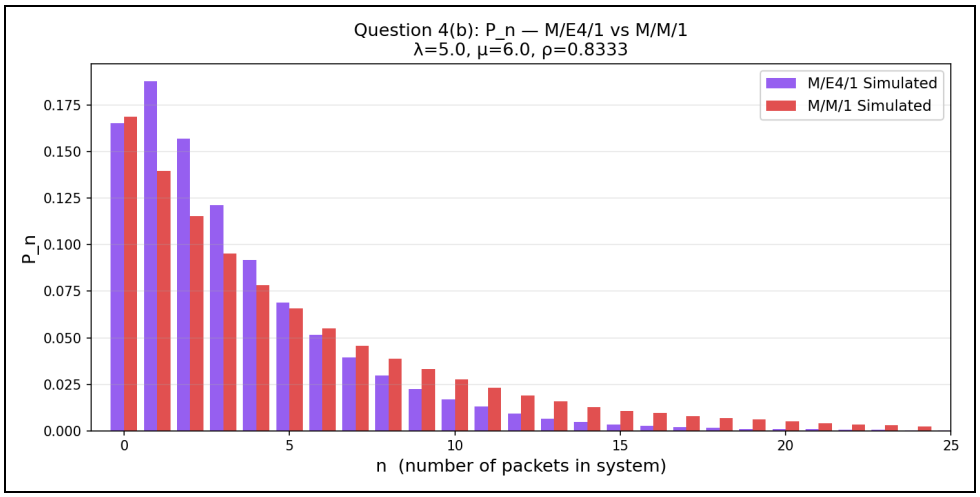
- Simulated E[N] for $M/E_4/1$: 3.4206
- $P_K$ value for $M/E_4/1$: 3.4375
- Simulated E[N] for M/M/1: 5.0739

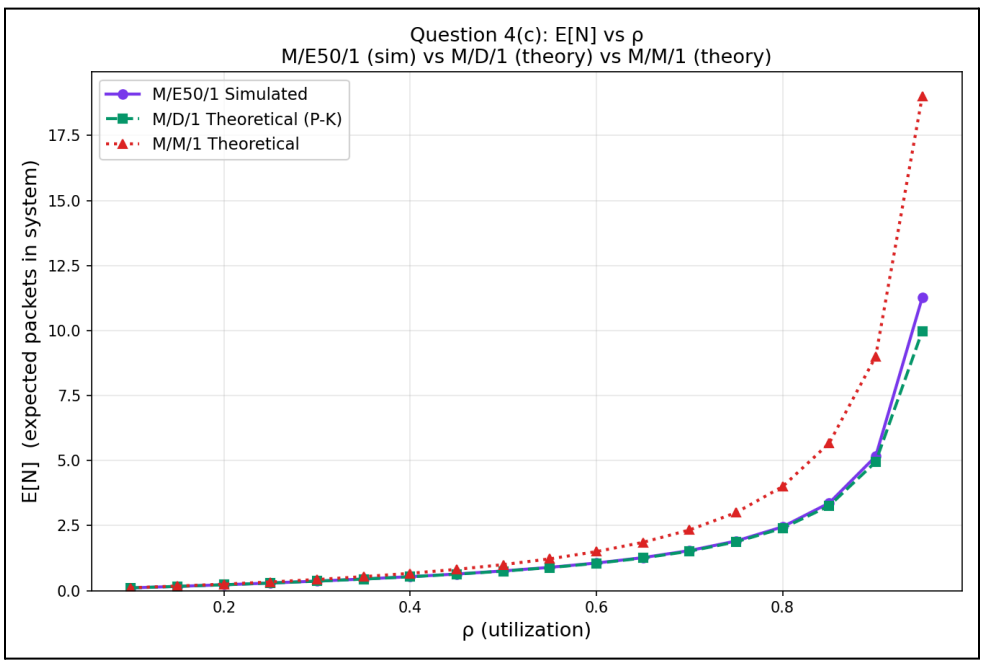
This is because Erlang service has lower variance than exponential service

For k = 50, the simulated $M/E_{50}/1$ results track M/D/1 closely, which shows that high phase Erlang service behaves nearly deterministically. For example:

- At $\rho = 0.50$, simulated $M/E_{50}/1$ gave about 0.757 while M/D/1 gave 0.75
- At $\rho = 0.90$, simulated $M/E_{50}/1$ gave about 5.163 while M/D/1 gave 4.95

Matplotlib Figures:




Notes:

- Lower service time variance reduces congestion
- $M/E_K/1$ moves closer to M/D/1 as k increases
- Both $M/E_{50}/1$ and M/D/1 stay below M/M/1 in expected system size for the same load

**Conclusions**

The project results are consistent with queuing theory and confirm the expected trends:

- Uniform samples can be used to construct Exponential, Poisson, and Erlang random variables
- Event driven simulation reproduces M/M/1 theoretical behavior accurately
- Reducing service time variability lowers average congestion
- As k becomes large, $M/E_K/1$ approaches M/D/1

**References**


Kleinrock, L. Queueing Systems, Volume 1: Theory. Wiley, 1975.

https://ia601403.us.archive.org/13/items

Used for M/M/1, M/G/1, and M/D/1 queueing formulas used to verify simulation output.

Gross, D., Shortle, J., Thompson, J., and Harris, C. Fundamentals of Queueing Theory. Wiley.

https://download.e-bookshelf.de/download/

Used for queueing background and steady state interpretation of simulation results.

Knuth, D. E. The Art of Computer Programming, Volume 2: Seminumerical Algorithms. Addison-Wesley, 1997.

https://seriouscomputerist.atariverse.com/media/pdf/

Used for the Poisson random variable generation method based on products of uniform random variables.

Devroye, L. Non-Uniform Random Variate Generation. Springer, 1986.

https://www.cs.fsu.edu/~mascagni/Devroye_Non-uniform_Abridged_1986.pdf

Used for inverse transform sampling for exponential random variables.